# pandas II: cleaning the mess, with a log and a reconciled total

*How do we make a raw export trustworthy, and prove what we changed?*

AI and Data Science Professional | Session 8 of 24 | Module B: Python for data

KLAi Professional Learning | Jotham Martin Wambi

# Today in one view: what you will leave with

## By the end of today you can



Remove duplicates, convert data types and dates, and clean text, each step with a stated rule

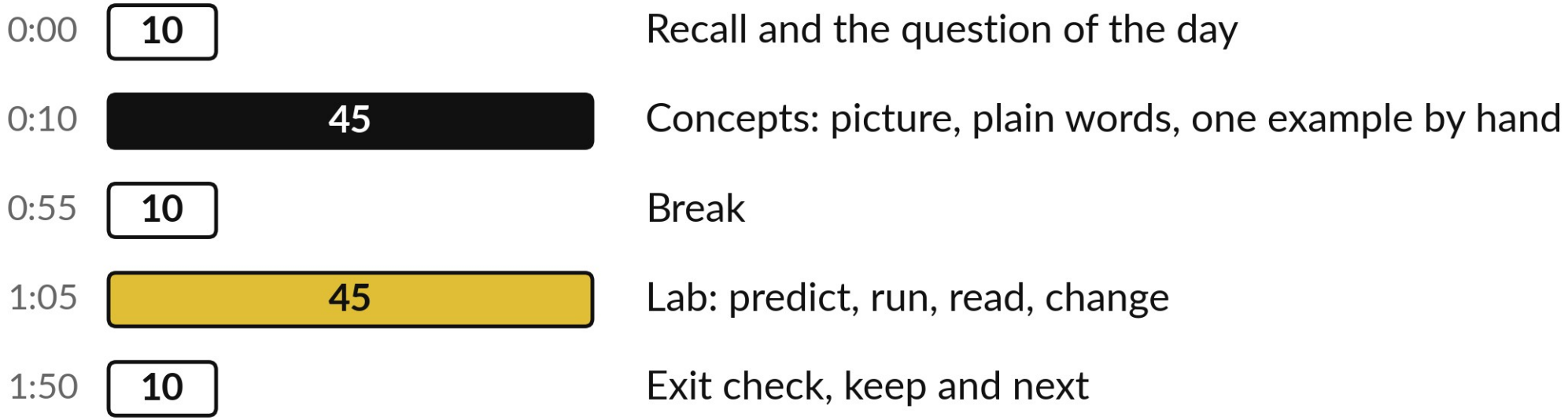


Handle missing values by dropping, imputing or flagging, and check outliers with the IQR rule before removing anything



Write a data cleaning log and reconcile the cleaned total to a control total

# Your two hours: concepts first, then the Project milestone 1 lab




Ideas with pictures first. Then you clean the file in the notebook. Then the checks and what you keep.

# Three questions from last time

1. Which line counts the empty cells in each column of a DataFrame?

2. How do you filter rows on two conditions?

3. You report a figure from a filtered table. What goes beside it?




Answer aloud first. Then we check together.

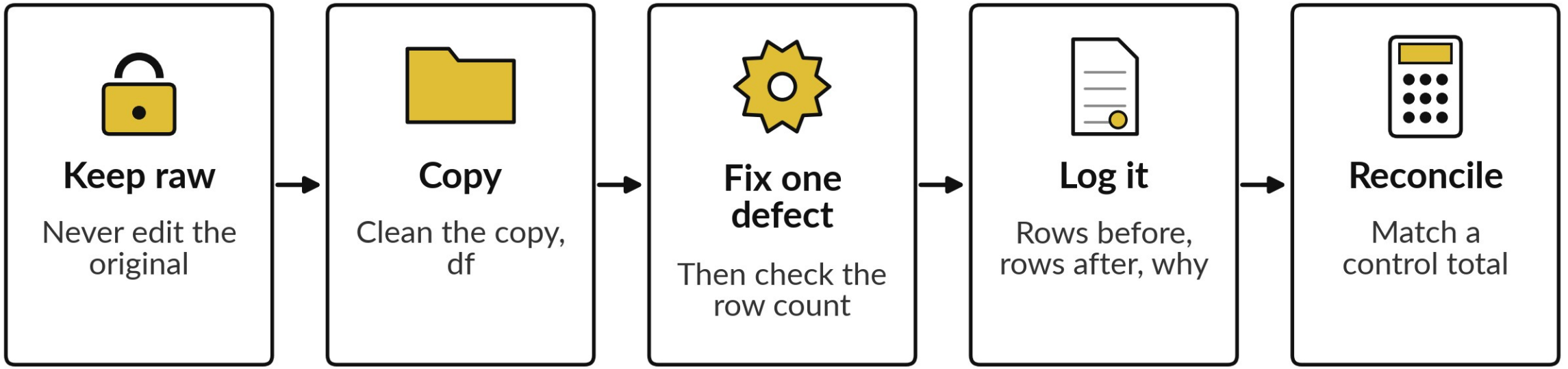
# Answers to the three questions from Session 7

| Question | Short answer |
|---|---|
| **Which line counts the empty cells in each column of a DataFrame?** | orders.isna().sum(): isna marks each empty cell True, and sum adds the Trues for each column. |
| **How do you filter rows on two conditions?** | Put each test in round brackets and join them with & for and, or \| for or. |
| **You report a figure from a filtered table. What goes beside it?** | The row count, so the reader knows how many rows the figure covers. |

# Words we use today

| | Word | What it means |
|---|---|---|
|  | **Data cleaning log** | A table of every change to the data, with the row counts before and after and the reason. It is the audit trail of the cleaning. |
|  | **Imputation** | Filling a missing value with a stated value, such as the median or a constant. Keep a flag to show which values were filled. |
|  | **Outlier (IQR rule)** | A value far from the rest. The IQR rule flags values more than 1.5 times the interquartile range beyond the quartiles. |
|  | **Control total** | A total that comes from a trusted source, such as the till system, used to test your own total. |
|  | **Reconciliation difference** | The gap between your cleaned total and the control total. Explain it, or find the defect behind it. |

# Cleaning is a workflow: keep the raw file, fix one defect at a time, log it, then reconcile




Data cleaning means making a file correct and consistent enough to analyse. Each change is small, checked and written down.

# Eight defects are hidden in the 41,259 lines of the raw export

| Defect | What you see in the raw file | Fix |
|---|---|---|
| **Exact duplicates** | 80 lines repeat another line | drop_duplicates |
| **Date as text, two formats** | 12,410 dates like 04/09/2025 | to_datetime with a format |
| **Price as text** | 10,345 prices like UGX 7,800 | to_numeric |
| **Payment spellings** | 11 spellings of 3 methods | String methods |
| **Branch spellings** | 26 spellings of 8 branches | String methods |
| **Blank customer** | 10,647 blanks in 3 spellings | Flag, then fill |
| **Blank quantity (qty)** | 12 lines | Drop, and explain |
| **Returns** | 1,181 negative quantities | Check, then keep |

Counts are for the raw file. After the 80 duplicates are dropped: slash dates 12,383, text prices 10,328, blank customers 10,626, negative quantities 1,179.

# read_csv turns N/A and NULL into NaN, so three blank spellings are already one

```
t = pd.read_csv('orders_raw.csv', dtype=str, keep_default_na=False)
t['customer_id'].value_counts().head(4)
```

```
customer_id
             5051
N/A          3113
NULL         2483
walk-in      1833
```

Read as plain text: three spellings of a blank.

```
raw = pd.read_csv('orders_raw.csv')
raw['customer_id'].isna().sum()
```

```
10647
```

The default read turns all three into NaN: 5,051 + 3,113 + 2,483.

The 1,833 walk-in labels stay as text. value_counts(dropna=False) counts the NaN too.

# An exact duplicate repeats another record in every variable, and drop_duplicates keeps the first

**Before: 41,259 lines**

| line_id | order_id | qty | unit_price |
|---|---|---|---|
| L002139 | O000669 | 1.0 | 7000 |
| L002139 | O000669 | 1.0 | 7000 |
| L002442 | O000760 | 2.0 | 3600 |
| L002442 | O000760 | 2.0 | 3600 |

**After: 41,179 lines**

| line_id | order_id | qty | unit_price |
|---|---|---|---|
| L002139 | O000669 | 1.0 | 7000 |
| L002442 | O000760 | 2.0 | 3600 |

```
df = raw.drop_duplicates()
df["line_id"].is_unique
```

**80 duplicates removed. After the cleaning line_id is unique: True. line_id is the primary key.**

Excel: Data, Remove Duplicates. pandas: duplicated(subset=...) checks chosen variables only.

# to_numeric converts text to numbers, and errors decides what happens to a bad value

## errors raise (the default)

Stops at the first bad value and names it.

Safe: you see the problem.

## errors coerce

Turns each bad value into NaN and carries on.

Here: 10,328 silent NaN values.

Risky: no warning.

**Remove the text first, convert, then count the NaN.**

# Strip the letters and the comma, then to_numeric: object becomes int64

**Before: unit_price is object (text)**

| line_id | unit_price |
|---|---|
| L068031 | UGX 7,800 |
| L084580 | 7000 |



**After: unit_price is int64**

| line_id | unit_price |
|---|---|
| L068031 | 7800 |
| L084580 | 7000 |

```
p = df['unit_price'].str.replace('UGX', '')
p = p.str.replace(',', '')
df['unit_price'] = pd.to_numeric(p)
```

**10,328 prices had the UGX text. After the conversion the type is int64 and no value is NaN.**

A price stored as text cannot be added up. A number can.

# to_datetime with a format reads each date style on its own, then fillna joins the two

```
d = df['order_date']
iso = pd.to_datetime(d, format='%Y-%m-%d', errors='coerce')
uk = pd.to_datetime(d, format='%d/%m/%Y', errors='coerce')
print(iso.isna().sum(), uk.isna().sum())
```

```
12383 28796
```

12,383 dates do not fit the year-month-day format, and 28,796 do not fit day/month/year.

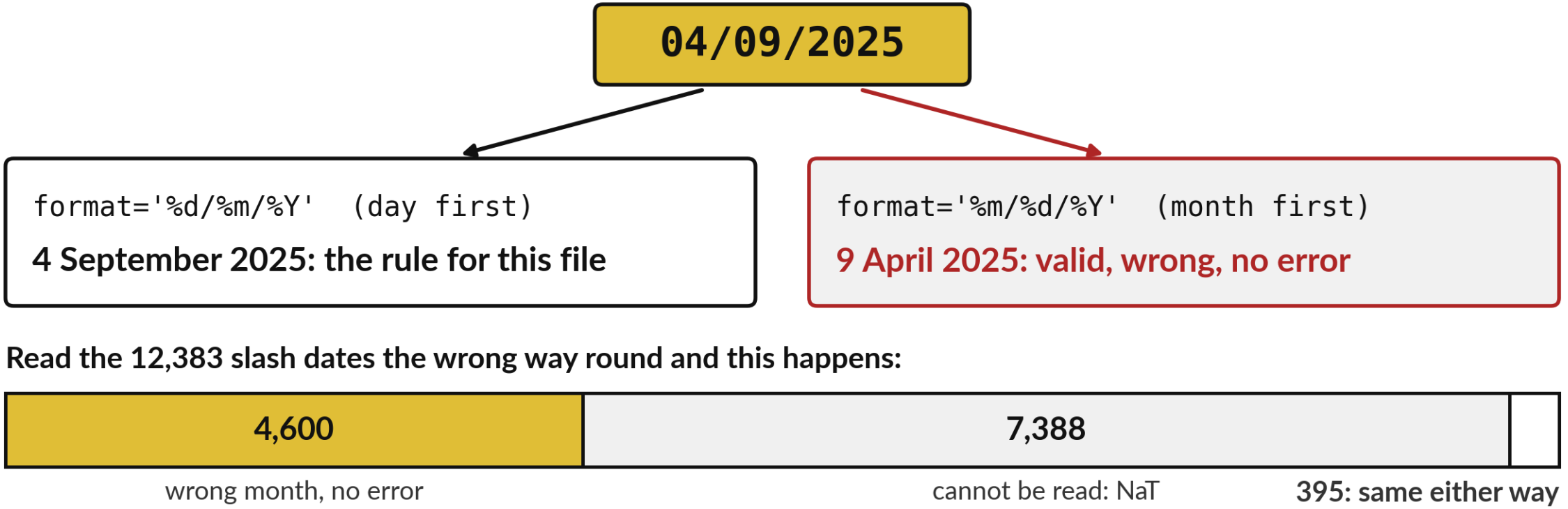
```
df['order_date'] = iso.fillna(uk)
d = df['order_date']
print(d.isna().sum(), d.min().date(), d.max().date())
```

```
0 2024-01-01 2025-12-28
```

No date is missing. The range is the two years of the file.

%Y is the year, %m the month, %d the day. NaT means not a time: a date that failed.

# Reading day/month dates the wrong way round raises no error, so learn the rule first




Ask the data owner which date style the file uses, and write it in the log.

# String methods fix text one rule at a time

| Method | What it does | Real example from the file |
|---|---|---|
| **str.strip** | Removes spaces at both ends | a trailing space after Cash |
| **str.lower** | Makes every letter lower case | CASH and cash become cash |
| **str.title** | Capital letter on each word | KAMPALA NTINDA becomes Kampala Ntinda |
| **replace with a dictionary** | Swaps each listed value | MM becomes mobile money, Entebe becomes Entebbe |
| **str.capitalize** | Capital letter on the first word | mobile money becomes Mobile money |

Excel twins: TRIM, LOWER, PROPER and SUBSTITUTE.

# The same rules cut 11 payment spellings to 3 and 26 branch spellings to 8

```
print(raw['payment'].nunique(), raw['branch'].nunique())
```

```
11 26
```

Different values before cleaning: payment, then branch.

```
b = df['branch'].str.strip().str.title()
fix = {'Kampala Cntral': 'Kampala Central', 'Entebe': 'Entebbe'}
df['branch'] = b.replace(fix)
pay = df['payment'].str.strip().str.lower()
df['payment'] = pay.replace({'mm': 'mobile money'}).str.capitalize()
print(df['payment'].nunique(), df['branch'].nunique())
```

```
3 8
```

After cleaning: three payment methods and eight branches.

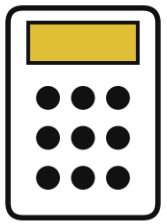
Check with value_counts that every remaining spelling is one you expect.

# A missing value has three treatments: drop, impute or flag



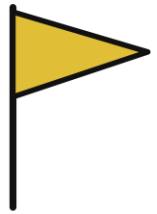
**Drop**

Remove the record. Use it when the record is useless without the value.



**Impute**

Fill with a stated value: the median, the mean or a constant.



**Flag**

Add an indicator variable that says the value was missing.

**First ask what the blank means. Then choose, and write the rule in the log.**

Drop: a sale with no quantity (qty) cannot be priced.

Impute: a constant such as walk-in, or the median of a number.

Flag: keeps the memory that the value was missing.

# Flag first, then fill: 10,626 blank customers become walk-in, and the flag remembers them

```
df['customer_missing'] = df['customer_id'].isna()
df['customer_id'] = df['customer_id'].fillna('walk-in')
print(df['customer_missing'].sum(), (df['customer_id'] == 'walk-in').sum())
```

```
10626 12456
```

10,626 flagged.
12,456 walk-in lines in all.

The flag is an indicator variable: True where the value was missing before we filled it.

# dropna with no settings would delete 26 percent of the lines, so name the column

```
len(df)
len(df.dropna())
```

```
41179
30541
```

Any blank cell removes the line: 10,638 lines gone, with their revenue.

```
len(df.dropna(subset=['qty']))
```

```
41167
```

Only the 12 lines with a blank qty are dropped.

Run these before the customer step. Print the row count before and after every drop.

# Median imputation fills a blank number, and an indicator variable keeps the memory

**Before: income is missing**

| loan_id | monthly_income_ugx |
|---|---|
| N00006 | NaN |
| N00012 | NaN |
| N00025 | NaN |

**After: filled with the median**

| loan_id | monthly_income_ugx | income_missing |
|---|---|---|
| N00006 | 642,600 | True |
| N00012 | 642,600 | True |
| N00025 | 642,600 | True |

```
median = loans['monthly_income_ugx'].median()
loans['income_missing'] = loans['monthly_income_ugx'].isna()
loans['monthly_income_ugx'] = loans['monthly_income_ugx'].fillna(median)
```

**loans.csv: 379 of 8,000 loans had no income. Median income: UGX 642,600.**

The median resists extreme values. We do not impute quantities, which would change revenue.

# The IQR rule flags a value beyond 1.5 interquartile ranges from the quartiles

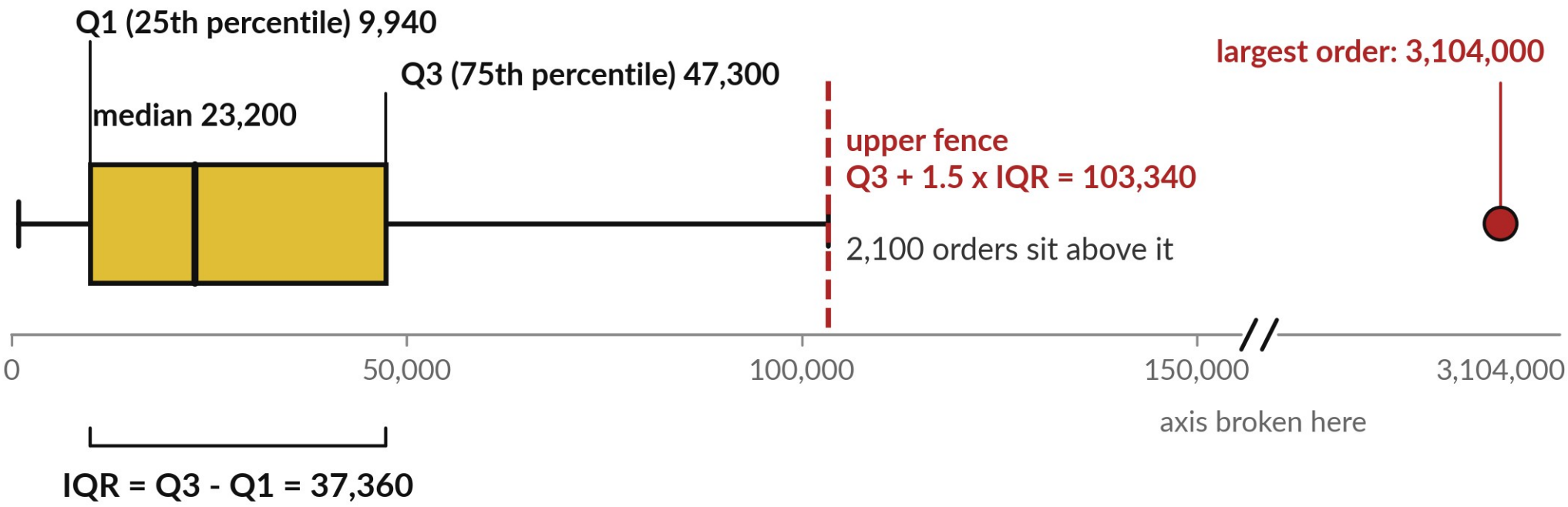
IQR rule $\quad \text{IQR} = Q_3 - Q_1 \qquad \text{fences} = Q_1 - 1.5\,\text{IQR},\ Q_3 + 1.5\,\text{IQR}$

Our order values $\quad 47,300 - 9,940 = 37,360 \qquad \text{upper fence} = 103,340$

| Symbol | What it means | In our data |
|---|---|---|
| **Q1** | First quartile: 25 percent of the values are smaller. | 9,940 UGX |
| **Q3** | Third quartile: 75 percent of the values are smaller. | 47,300 UGX |
| **IQR** | Interquartile range: the spread of the middle half. | 37,360 UGX |
| **Fences** | Values outside them are flagged as outliers. | lower -46,100, upper 103,340 |

A negative lower fence means no value can be flagged on the low side.

# The IQR rule flags 2,100 of 30,229 orders, and each flag is a question to ask




Order values from orders.csv, the full file. The gold box is the middle half of the orders.

# Check an outlier before you remove it: real, error or a different kind of record

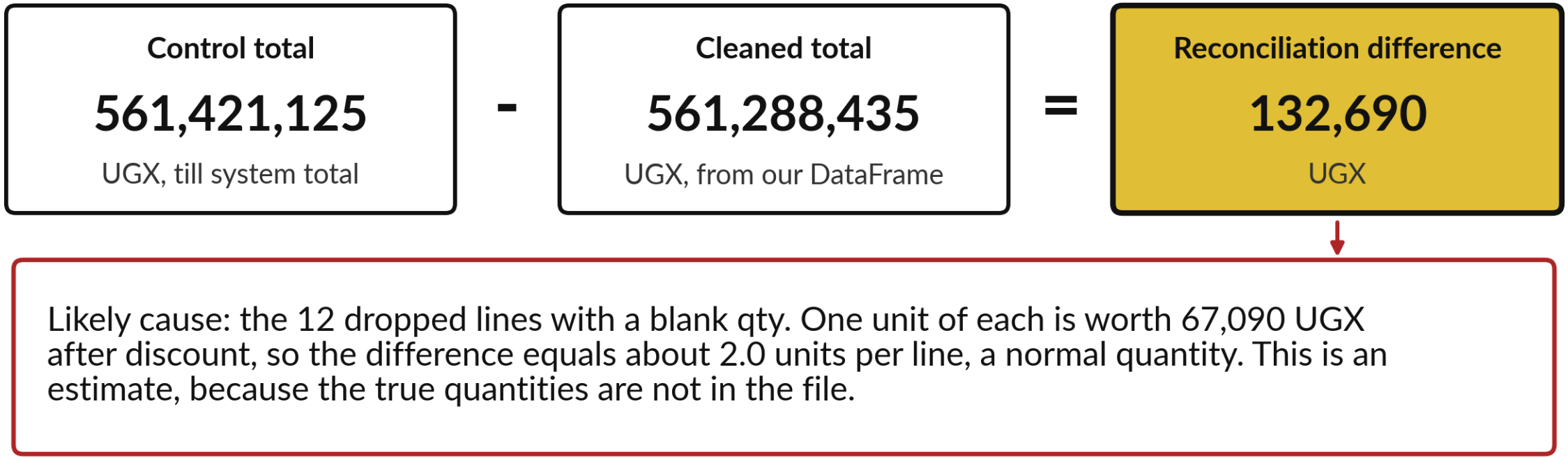
| | Case | Evidence from our files | Action |
|---|---|---|---|
|  | **A real extreme** | Largest order 3,104,000 UGX: possible, so ask the data owner. | Keep. Note it in the log. |
|  | **An impossible value** | loans.csv: ages 0 and 199 among 8,000 loans. The IQR rule flags 25 ages, only 2 impossible. | Set to missing, then treat as missing. |
|  | **A different kind of record** | 1,179 negative quantities: returns, with line_id ending in R. | Keep. They reduce revenue. |
|  | **A value that should match a list** | Prices differing from the price list: 0. | Nothing to fix. Log the check. |

# The data cleaning log records every change with its row counts and its reason

| Step | Rows before | Rows after | Why |
|---|---|---|---|
| **Drop exact duplicates** | 41,259 | 41,179 | 80 identical lines |
| **Price text to number** | 41,179 | 41,179 | 10,328 prices such as UGX 7,800 |
| **Two date formats to dates** | 41,179 | 41,179 | 12,383 day/month/year dates |
| **Branch and payment text** | 41,179 | 41,179 | 26 branch spellings to 8, 11 payment spellings to 3 |
| **Flag and fill blank customers** | 41,179 | 41,179 | 10,626 blanks flagged, then labelled walk-in |
| **Drop lines with blank qty** | 41,179 | 41,167 | 12 lines: the quantity cannot be recovered |
| **Check outliers, keep returns** | 41,167 | 41,167 | 1,179 returns kept. IQR rule on revenue: 2,130 high and 385 low lines, all kept |

Equal counts: a step changed values and left the rows alone.

# Reconcile: the cleaned total is compared with the control total, and the gap is explained




Returns are already inside the cleaned total as negative lines.

# The reconciliation difference is the control total minus the cleaned total

Definition

$$\text{difference} = \text{control total} - \text{cleaned total}$$

Our data

$$561,421,125 - 561,288,435 = 132,690$$

| Symbol | What it means | In our data |
|---|---|---|
| **Control total** | A total from a trusted source, here the till system. | 561,421,125 UGX |
| **Cleaned total** | Your total after cleaning, from the code. | 561,288,435 UGX |
| **Difference** | What is left unexplained. It must be explained, or it is a defect. | 132,690 UGX |

Use the name reconciliation difference. The word residual belongs to models, from Session 13.

# Project milestone 1: what you produce and what a finished version contains

| | You produce | A finished version has |
|---|---|---|
|  | **The cleaned notebook** | Every check cell says OK and the run ends with All checks pass. |
|  | **The data cleaning log** | It has one row for each of the seven steps, and the last row ends at 41,167. |
|  | **A reconciliation note** | One or two sentences name the reconciliation difference (132,690 UGX) and its likely cause. |
|  | **The files orders_clean.csv and cleaning_log.csv** | They are saved from df and the log, and open in Excel or Sheets. |

The log is saved as cleaning_log.csv. Your tutor reviews it with you for feedback. It is never scored.

# Where you will use this at work: clean, log and reconcile every export

| | Your job | The mess you meet | What you do with today |
|---|---|---|---|
|  | **Credit analyst at a bank, with pandas** | loans.csv: 379 blank incomes, ages 0 and 199. | Impute with a flag, set impossible ages to missing, and log both. |
|  | **Finance officer reconciling till exports** | A till export with 80 duplicate lines and prices stored as text. | Drop duplicates, convert prices, and reconcile it to the end-of-day control total. |
|  | **Health records officer at a district office** | Clinic file: 18 blank months out of 720. | Flag the blank months instead of treating them as zero. |
|  | **SACCO officer, using Excel** | Member payment exports with several spellings. | Write the cleaning rules once and rerun them on each new export. |

# Read the recipe before the lab: copy, drop duplicates, convert the price

```
df = raw.copy()
df = df.drop_duplicates()
p = df['unit_price'].str.replace('UGX', '')
p = p.str.replace(',', '')
df['unit_price'] = pd.to_numeric(p)
print(df['unit_price'].dtype, len(df))
```

```
int64 41179
```

## In plain words

Line 1 copies raw. We never edit raw.

Line 2 drops 80 duplicate lines.

Lines 3 and 4 remove the text UGX and the comma.

Line 5 converts the text to numbers.

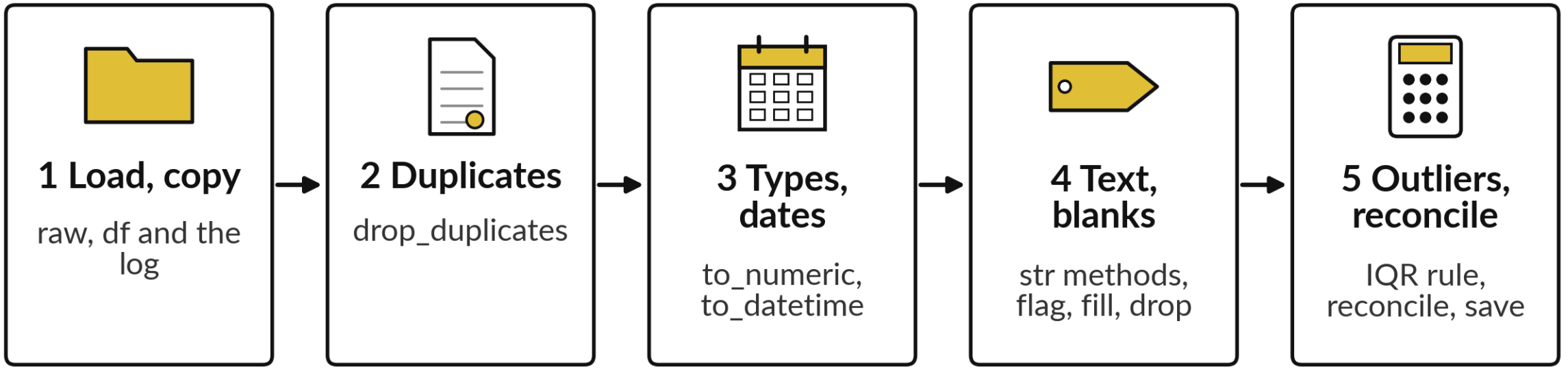
Line 6 prints the type, int64, and the row count.

KLAI

# Part 2: the cleaning lab and Project milestone 1

Close the slides. Open the notebook.

# The lab has eight steps: seven cleaning steps and the reconciliation




Eight steps in 45 minutes, grouped in five boxes. For each step: predict, run, read, change one thing, and add a log row.

# Check your numbers before you move on

| What you compute | You should get |
|---|---|
| **Shape of raw** | (41,259, 10) |
| **Duplicates removed, rows after** | 80, 41,179 |
| **Prices with UGX text, dtype after to_numeric** | 10,328, int64 |
| **Dates missing after the two formats, earliest and latest** | 0, 2024-01-01 and 2025-12-28 |
| **Different branches and payment methods after cleaning** | 8 and 3 |
| **Blank customers flagged, walk-in lines after filling** | 10,626 and 12,456 |
| **Lines with blank qty dropped, rows left** | 12, 41,167 |
| **Cleaned total and reconciliation difference** | 561,288,435 and 132,690 |

If your number differs, find the step that went wrong. Do not change a number to match.

# Five likely mistakes in the lab and how to fix them

| What you see | What it means | What to do |
|---|---|---|
| **ValueError: Unable to parse string** | Text is left in the price, so to_numeric cannot read it. | Remove UGX and the comma first, then convert. |
| **Dates still missing after the fill** | A format string does not match the text. | Match each format to its separator: dash or slash. |
| **AttributeError: Can only use .str accessor** | The column is already a number or a date. | Run each string method before the conversion. |
| **Row count drops by about ten thousand** | dropna was called with no settings. | Use dropna with subset set to the column qty. |
| **Your net total is a little different** | A step ran twice, or the returns were removed. | Restart and run all cells from the top. Keep the returns. |

# Exit check: three questions, then compare with the short answers

| | Question | Short answer |
|---|---|---|
|  | **Why should you flag a blank before you fill it?** | The flag is an indicator variable that remembers which values were filled. After filling, you can no longer tell. |
|  | **The IQR rule flags 25 of 8,000 loan ages. Do you delete them?** | No. A flag is a question. Only the impossible ages, 0 and 199, are errors. The rest are real borrowers. |
|  | **Your cleaned total is 132,690 below the control total. What do you write?** | The reconciliation difference is 132,690 UGX. Likely cause: the 12 dropped lines with a blank qty, about 2 units each. It is an estimate, because the true quantities are not in the file. |

# Save your notebook, orders_clean.csv and the log

## What you keep from today

Keep your notebook, the file orders_clean.csv, and your data cleaning log with its reconciliation note: a file you can hand to a manager with the proof of what you changed.

### Our question today

*How do we make a raw export trustworthy, and prove what we changed?*

### Next session

Session 9: pandas III, group, join and reshape. We answer questions that need several tables and several groups.

**Take away**

Your lab file

*orders_clean.csv*

The key proof

*Log and reconciled total*

Next session

*Group, join, reshape*